

```

NextionProtocol.cpp
//=====
//Begin of Nextion LCD Protocol
//
// v0~v9, va~vz : Numeric (Transceiver -> Nextion LCD)
// s0~s9 : String (Text) (Transceiver -> Nextion LCD)
// vlSendxxx, vloxxx: Reserve for Nextion (Nextion LCD -> Transceiver)
//
//=====

#include <arduino.h>
#include "Configuration.h"
#include "i2cmeter2.h"
#include "NextionProtocol.h"
// #include "AltSoftSerial.h"
#include "fftfuctions.h"
#include <EEPROM.h>

void NextionProtocol::Initialize()
{
    //Load Configuration
    EEPROM.get(EEPROM_DSPTYPE, DSPTYPE);
    if (DSPTYPE > 5)
    {
        DSPTYPE = 1;
    }

    //Signal Meter
    EEPROM.get(EEPROM_SMETER_UART, SMeterToUartSend);
    if (SMeterToUartSend > 2)
    {
        SMeterToUartSend = 1;
    }
    //
    EEPROM.get(EEPROM_CW_FREQ, cwDecodeHz);
    if (cwDecodeHz > 40 || cwDecodeHz < 1)
    {
        cwDecodeHz = 9;
    }

    //EEPROM_CW_MAG_LOW
    EEPROM.get(EEPROM_CW_MAG_LOW, magnitudelimit_low);
    if (magnitudelimit_low > 1000 || magnitudelimit_low < 1)
    {
        magnitudelimit_low = 50;
    }
}

```

```

    coeff = CalculateCoeff(cwDecodeHz + 6, DECODE_MORSE_SAMPLESIZE); //Set 750Hz //9 *
50 + 300 = 750Hz // add 6 because nextion has a ~300 Hz offset
}

//send data for Nextion LCD
void NextionProtocol::SendHeader(char varType, char varIndex)
{
    SERIAL_OUTPUT.print(F("pm."));
    SERIAL_OUTPUT.write(varType == SWS_HEADER_STR_TYPE ? 's' : 'v');
    SERIAL_OUTPUT.write(varIndex);

    if (varType == SWS_HEADER_STR_TYPE)
    {
        SERIAL_OUTPUT.print(F(".txt=\""));
    }
    else
    {
        SERIAL_OUTPUT.print(F(".val="));
    }
}

void NextionProtocol::SendCommand1Num(char varIndex, char sendValue) //0~9 : Mode,
nowDisp, ActiveVF0, IsDialLock, IsTxtType, IsSplitType
{
    char softTemp[12] = { 0 };
    ultoa(sendValue, softTemp, DEC);

    SERIAL_OUTPUT.print(F("pm.c"));
    SERIAL_OUTPUT.write(varIndex);
    SERIAL_OUTPUT.print(F(".val="));
    SERIAL_OUTPUT.write(softTemp);
    SendCommandEnding(varIndex);
}

void NextionProtocol::SendCommandUL(char varIndex, unsigned long sendValue)
{
    SendHeader(SWS_HEADER_INT_TYPE, varIndex);

    char softTemp[12] = { 0 };
    ultoa(sendValue, softTemp, DEC);
    SERIAL_OUTPUT.print(softTemp);
    SendCommandEnding(varIndex);
}

void NextionProtocol::SendCommandL(char varIndex, long sendValue)
{
    SendHeader(SWS_HEADER_INT_TYPE, varIndex);

```

```

    char softTemp[12] = { 0 };
    ltoa(sendValue, softTemp, DEC);
    SERIAL_OUTPUT.print.softTemp);
    SendCommandEnding(varIndex);
}

void NextionProtocol::SendCommandStr(char varIndex, char* sendValue)
{
    SendHeader(SWS_HEADER_STR_TYPE, varIndex);

    SERIAL_OUTPUT.print(sendValue);
    SendCommandStrEnding(varIndex);
}

void NextionProtocol::SendCommandEnding(char varIndex)
{
    SERIAL_OUTPUT.print(F("\xFF\xFF\xFF"));
    lastForwardmili = millis();
}

void NextionProtocol::SendCommandStrEnding(char varIndex)
{
    SERIAL_OUTPUT.write('\n');
    SendCommandEnding(varIndex);
}

//Result : if found .val=, 1 else 0
void NextionProtocol::CommandParser(uint8_t* ForwardBuff, int lastIndex)
// void NextionProtocol::CommandParser(char* ForwardBuff, int lastIndex)
{
    //Analysing Forwrd data
    //59 58 68 4A 1C 5F 6A E5 FF FF 73
    //Find Loopback protocol
    // 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
    //70 6D 2E 76 76 2E 76 61 6C 3D 33 38 34 38 39 35 33 36 32 38 FF FF FF
    //pm.vv.val=3848953628\xFF\xFF\xFF
    //1234567890XXX
    //
    int startIndex = 0;

    // Return fast if not a loopback
    // Loop back command has 13 ~ 23
    if (lastIndex < 13 || lastIndex >= MAX_FORWARD_BUFF_LENGTH)
    {
        return;
    }

    for (int i = lastIndex - 3; i >= startIndex + 7; i--)

```

```

{
    //Find =
    if (ForwardBuff[i-3] == 'v' && ForwardBuff[i-2] == 'a' && ForwardBuff[i-1] == '\l'
&& ForwardBuff[i] == '=') //0x3D
    {
        uint8_t command1 = ForwardBuff[i-6]; //v
        uint8_t command2 = ForwardBuff[i-5]; //v
        // i-4 //.
        ForwardBuff[lastIndex - 2] = 0;

        long commandVal=atol((const char*) &ForwardBuff[i + 1]);
        // long commandVal=atol(&ForwardBuff[i + 1]);
        uint8_t *ReadBuff = (uint8_t *)&commandVal;

        //Loop Back
        if (command1 == 'v' && command2 == 'v')
        {
            //=====
            //Test Code 1
            /*
            long tmpVal = 0;
            SERIAL_OUTPUT.print("Found :");
            for (int k = i + 1; k <= lastIndex - 3; k++)
            {
                SERIAL_OUTPUT.write(ForwardBuff[k]);
            }
            SERIAL_OUTPUT.println();
            SERIAL_OUTPUT.print("Reverse :");
            for (int k = lastIndex - 3; k >= i + 1; k--)
            {
                SERIAL_OUTPUT.write(ForwardBuff[k]);
            }
            SERIAL_OUTPUT.println();
            ForwardBuff[lastIndex - 2] = 0;
            tmpVal=atol(&ForwardBuff[i + 1]);
            SERIAL_OUTPUT.println(tmpVal);
            uint8_t *ReadBuff = (uint8_t *)&tmpVal;
            char buff[10];
            sprintf(buff, "%x,%x,%x,%x", ReadBuff[0], ReadBuff[1], ReadBuff[2],
ReadBuff[3]);
            SERIAL_OUTPUT.println(buff);
            int calcChecksumA = ReadBuff[0] + ReadBuff[1] + ReadBuff[2];
            calcChecksumA = calcChecksumA % 256;
            if (calcChecksumA == ReadBuff[3])
            {
                SERIAL_OUTPUT.print("Correct Checksum : ");
                SERIAL_OUTPUT.print(ReadBuff[3]);
                SERIAL_OUTPUT.print(":");
            }
        }
    }
}

```

```

    SERIAL_OUTPUT.println(calcChecksumA);
}
//End of Test Code
*/

/*
//Test Code 2
SERIAL_OUTPUT.print("Found :");
SERIAL_OUTPUT.print(startIndex);
SERIAL_OUTPUT.print(",");
SERIAL_OUTPUT.print(lastIndex);
SERIAL_OUTPUT.print(":");
for (int k = i + 1; k <= lastIndex - 3; k++)
{
    SERIAL_OUTPUT.write(ForwardBuff[k]);
}
SERIAL_OUTPUT.println("");
//End of Tet Code 2
*/

int calcChecksum = ReadBuff[0] + ReadBuff[1] + ReadBuff[2];
calcChecksum = calcChecksum % 256;

//Correct Checksum and Receiver is DSP Moudle protocol v1.0
if (calcChecksum == ReadBuff[3] && ReadBuff[2] == 0x6A)
{
    //SERIAL_OUTPUT.print("Correct Checksum Command : ");
    //SERIAL_OUTPUT.println(ReadBuff[1]);
    uint8_t cmd1 = ReadBuff[1];
    if (cmd1 == 94)
    {
        DSPTYPE = 0;
        EEPROM.put(EEPROM_DSPTYPE, DSPTYPE);
    }
    else if (cmd1 == 95)
    {
        //SERIAL_OUTPUT.println("Spectrum Mode");
        DSPTYPE = 1;
        EEPROM.put(EEPROM_DSPTYPE, DSPTYPE);
    }
    else if (cmd1 >= 100 && cmd1 <= 145)
    {
        cwDecodeHz = cmd1 - 100;
        coeff = CalculateCoeff(cwDecodeHz + 6 , DECODE_MORSE_SAMPLESIZE); // add
6 because nextion has a ~300 Hz offset
        DSPTYPE = 2;
        EEPROM.put(EEPROM_DSPTYPE, DSPTYPE);
        EEPROM.put(EEPROM_CW_FREQ, cwDecodeHz);
    }
}

```

```

    }
    else if (cmd1 > 1 && cmd1 <= 5) //2~5 : Request Configuration
    {
        responseCommand = cmd1;
    }
    else if (cmd1 == 50 || cmd1 == 51) //Set Configuration
    {
        SMeterToUartSend = (cmd1 == 51);
        EEPROM.put(EEPROM_SMETER_UART, SMeterToUartSend);
    }
    else if (cmd1 >= 146 && cmd1 <= 156 )
    {
        //Save Mode
        magnitudelimit_low = (cmd1 - 146) * 10;
        EEPROM.put(EEPROM_CW_MAG_LOW, magnitudelimit_low);
    } //end of if
} //end of check Checksum
} //end of check Protocol (vv)
else if (command1 == 'c' && command2 == 't') //TX, RX
{
    if (commandVal == 0) //RX
    {
        TXStatus = 0;
        SMeterToUartIdleCount = 0;
    }
    else if (commandVal == 1) //TX
    {
        TXStatus = 1;
        SMeterToUartIdleCount = 0;
    }
}
else if (command1 == CMD_VFO_TYPE && command2 == CMD_CURR_FREQ) // WC8C - Save
frequency for use in power meter calibration adjustments
{
    L_vfoCurr = commandVal;
    frequencyHasChanged = true;
}

return;
} //end of check Protocol (.val)
} //end of for

//Not found Protocol (.val=
return;
}

void NextionProtocol::ForwardData(void)
// int NextionProtocol::ForwardData(void)

```

```

{
    static uint8_t etxCnt = 0; // static so value is retained between executions
    static uint8_t nowBuffIndex = 0; // static so value is retained between executions
    static uint8_t ForwardBuff[MAX_FORWARD_BUFF_LENGTH + 1];

    uint8_t recvChar;

    while (Serial.available() > 0) // make sure all data has been forwarded before doing
    anything else.
    {
        if (Serial.available() > 0)
        {
            //Check RX Buffer
            while (Serial.available() > 0)
            {
                recvChar = Serial.read();
                SERIAL_OUTPUT.write(recvChar);

                if ( nowBuffIndex < MAX_FORWARD_BUFF_LENGTH )
                {
                    ForwardBuff[nowBuffIndex] = recvChar;
                    nowBuffIndex++;
                }

                if (recvChar == 0xFF) //found ETX
                {
                    etxCnt++; //Nextion Protocol, ETX : 0xFF, 0xFF, 0xFF
                    if (etxCnt >= 3)
                    {
                        nextionIsConnected = true; // Set to true first time through then stays
                        true. This indicates commands are coming in so assume a Nextion is connected.
                        //Finished Protocol
                        CommandParser(ForwardBuff, nowBuffIndex);
                        nowForwardingProtocol = false; //Finished 1 Set Command
                        etxCnt = 0;
                        nowBuffIndex = 0;
                    }
                }
            }
        }
        else
        {
            etxCnt = 0;
            nowForwardingProtocol = true; //Sending Data
        }
    } //end of while
    SERIAL_OUTPUT.flush();
    lastForwardmili = millis();
}

```

```

    }
}

void NextionProtocol::SendScaledSMeter(int meterToUartInterval)
{
    if (SMeterToUartSend == 1 && LastSendScaledSMeter != scaledSMeter &&
LastSendWaitTimeElapsed() && ++SMeterToUartIdleCount > meterToUartInterval)
    {
        SendCommand1Num(CMD_SMETER, scaledSMeter);
        SMeterToUartIdleCount = 0;
        LastSendScaledSMeter = scaledSMeter;
    }
}

void NextionProtocol::CalculateScaledSMeter(int ADC_DIFF)
{
    // if not in high resolution SMeter mode, this will multiply by 10 so that the IF
    // statements below work. For High Res, it multiplies by 1
    int HIGH_RESOLUTION_ADC_DIFF = ADC_DIFF * (10 / SMETER_RESOLUTION_MULTIPLIER);

    //basic : 1.5Khz
    uint8_t newScaledSMeter = 0;

    if (HIGH_RESOLUTION_ADC_DIFF >= 360) //s9 + 10dbm to s9 + 20dbm
    {
        newScaledSMeter = map(constrain(ADC_DIFF, 0, 1560), 360, 1560, 72, 80);
    }
    if (HIGH_RESOLUTION_ADC_DIFF >= 118) //s9 to s9 + 10dbm
    {
        newScaledSMeter = map(ADC_DIFF, 118, 360, 64, 72);
    }
    else if (HIGH_RESOLUTION_ADC_DIFF >= 64) //s8 to s9
    {
        newScaledSMeter = map(ADC_DIFF, 64, 118, 52, 64);
    }
    else if (HIGH_RESOLUTION_ADC_DIFF >= 40) // s7 to s8
    {
        newScaledSMeter = map(ADC_DIFF, 40, 64, 40, 52);
    }
    else
    {
        // Anything smaller is close enough to linear.
        newScaledSMeter = ADC_DIFF;
    }

    // if not in high resolution SMeter mode, undo the scaling
    newScaledSMeter = constrain(newScaledSMeter, 0, 80) / (10 /
SMETER_RESOLUTION_MULTIPLIER) ;
}

```



```

/*
1 : with noise (not use 0 ~ S3)
2 : -93 ~ -89
3 : -88 ~ -81
4 : -80 ~ -78
5 : -77 ~ -72
6 : -71 ~ -69
*/

noInterrupts();    // disable interrupts while saving these so that the I2C ISR cant
read them while they are changing.
scaledSMeter = newScaledSMeter;
interrupts();
}

bool NextionProtocol::ResponseConfig()
{
    bool response = false;
    if (responseCommand > 0 && LastSendWaitTimeElapsed())
    {
        if (responseCommand == 2)
        {
            unsigned long returnValue = 0;
            if (DSPTYPE == 0)
            {
                returnValue = 94; //None
            }
            else if (DSPTYPE == 1)
            {
                returnValue = 95; //Spectrum (FFT) mode
            }
            else if (DSPTYPE == 2)
            {
                returnValue = 100 + cwDecodeHz;
            }

            returnValue = returnValue << 8;
            returnValue = returnValue | (SMeterToUartSend & 0xFF);
            returnValue = returnValue << 8;
            uint8_t tmpValue = 0;
            if (magnitudelimit_low > 255)
                tmpValue = 255;
            else if (magnitudelimit_low < 1)
                tmpValue = 0;
            else
                tmpValue = magnitudelimit_low;
            returnValue = returnValue | (tmpValue & 0xFF);
        }
    }
}

```

```

        SendCommandUL('v', returnValue);    //Return data
        SendCommandUL('g', 0x6A);    //Return data
        response = true;
    }
    responseCommand = 0;
}
return response;
}

void NextionProtocol::SendFFTData(int readSampleCount, int *readArray)
{
    SendHeader(SWS_HEADER_STR_TYPE, CMD_SMETER);

    for(uint8_t i = 1; i < FFTSIZE; i++)
    {
        if (SerialDataToProcess())
        {
            // if something has come in then terminate the FFT send so it can be processed.
            // this is so that frequency tuning does no lag when in FFT mode.
            // this will be checked before each "expensive" operation
            break;
        }

        double freqCoeff = CalculateCoeff(i, SAMPLESIZE);

        uint8_t readedValue;
        if (!SerialDataToProcess())
        {
            readedValue =
constrain((int)GetMagnatude(freqCoeff, SAMPLESIZE, readArray), 0, 255);
        }

        if (!SerialDataToProcess())
        {
            char hexChar[3];
            itoa(readedValue >> 4, hexChar, HEX);
            SERIAL_OUTPUT.write(hexChar[0]);
            itoa(readedValue & 0xf, hexChar, HEX);
            SERIAL_OUTPUT.write(hexChar[0]);
        }
    }

    SendCommandStrEnding(CMD_SMETER);
}

bool NextionProtocol::SendPowerSwr(float power, float swr, bool sendSwrAsSmeter)

```

```

// bool NextionProtocol::SendPowerSwr(float power, float swr, bool sendSwrAsSmeter =
true)
{
    // Send SWR as S meter value. This is for comaptibility with Dr. Lee's version
    uint8_t SmeterSwrValue = constrain((uint8_t)(((swr - 1.0) *
SMETER_RESOLUTION_MULTIPLIER) + 0.5), 1, 9 * SMETER_RESOLUTION_MULTIPLIER); //
Subtract 1 so the SMeter is zero based.
    if (LastSendScaledSMeter != SmeterSwrValue && sendSwrAsSmeter)
    {
        if (WaitUntilCommandCanBeSent())
        {
            SendCommand1Num(CMD_SMETER, SmeterSwrValue);
            LastSendScaledSMeter = SmeterSwrValue;
        }
        else
        {
            return false; // Wait was aborted because data came in to process
        }
    }

    if (lastSentSwr != swr)
    {
        //SWR Send
        if (WaitUntilCommandCanBeSent())
        {
            SendCommandL(CMD_UBITX_INFO, constrain((int)((swr * 100.0)+.5),1 ,999)); //
shift decimal point because nextion only handles integers. Round value. Limit max
value to 999
            SendCommand1Num(CMD_UBITX_INFO, 3);
            lastSentSwr = swr;
        }
        else
        {
            return false; // Wait was aborted because data came in to process
        }
    }

    if (lastSentPower != power)
    {
        //PWR Send
        if (WaitUntilCommandCanBeSent())
        {
            SendCommandL(CMD_UBITX_INFO, (int)((power * 100.0)+ .5)); // shift decimal point
because nextion only handles integers. Round value.
            SendCommand1Num(CMD_UBITX_INFO, 2);
            lastSentPower = power;
        }
        else
    }

```

```

    {
        return false; // Wait was aborted because data came in to process
    }
}

return true;
}

bool NextionProtocol::WaitUntilCommandCanBeSent()
{
    // Pauses an interval to give the nextion time to process the last command
    // returns false if the wait was aborted, in which case the send should be aborted
    because there
    // is inbound data to process

    while (!LastSendWaitTimeElapsed() && !SerialDataToProcess());
    return !SerialDataToProcess();
}

bool NextionProtocol::SerialDataToProcess()
{
    return (Serial.available() > 0 || nowForwardingProtocol);
}

bool NextionProtocol::LastSendWaitTimeElapsed()
{
    return (millis() - lastForwardmili > LAST_TIME_INTERVAL);
}

bool NextionProtocol::SendDecodeCharacters(char *characters)
{
    if (LastSendWaitTimeElapsed() && characters[0] != 0)
    {
        SendCommandStr('b', characters);
        return true;
    }
    return false;
}

```